University *of Ljubljana*
Faculty *of Computer and Information Science*

Zakonodajko: Retrieval-Augmented Question Answering over Slovenian Tax Legislation

ByteBanda project team

Abstract

Zakonodajko is a domain-specific question-answering assistant for Slovenian tax legislation. The system uses Retrieval-Augmented Generation (RAG): legal documents are extracted from local PISRS HTML files, split into chunks, embedded with a multilingual sentence-transformer, indexed with FAISS, and retrieved as grounded context for a local instruction-tuned language model. We implemented two retrieval pipelines. The baseline uses fixed-size character chunks and dense FAISS retrieval. Version 2 adds article-aware legal chunking and a hybrid reranker that combines dense similarity, lexical overlap, and a legal-source boost. On a 20-question evaluation set covering DDV, dohodnina, corporate income tax, and tax procedure, the Version 2 retriever improves source hit@3 from 40.0% to 95.0%, article hit@3 from 0.0% to 30.0%, and expected phrase hit@3 from 0.0% to 35.0%. Manual generation assessment shows that the strict prompt is slightly more correct than the baseline prompt, but correctness remains limited at 0.65/2 on average because many failures originate from retrieval of related but not exact legal articles.

Keywords

retrieval-augmented generation, Slovenian legislation, tax law, FAISS, sentence embeddings

Advisor: Slavko Žitnik

Introduction

Legal question answering is a difficult application for general-purpose language models because answers must be grounded in precise and current source material. For tax questions, small differences in article number, exception, threshold, or legal act can change the answer. Our project, *Zakonodajko*, addresses this problem by building a Retrieval-Augmented Generation system for Slovenian tax law. Instead of relying only on model memory, the assistant first retrieves relevant legal passages and then generates an answer from those passages.

The current prototype focuses on Slovenian tax legislation from PISRS. The main source documents are the Value Added Tax Act (ZDDV-1), the Personal Income Tax Act (ZDoh-2), the Corporate Income Tax Act (ZDDPO-2), the Tax Procedure Act (ZDavP-2), and related rulebooks. These documents are suitable for a RAG system because they are long, structured into articles, and contain many definitions and procedural rules that users may ask about.

The project follows the standard RAG approach introduced by Lewis et al. [1]. We also considered more complex adaptation methods such as LoRA fine-tuning [2] and tool-using language models [3], but for this stage we chose a transparent retrieval-first pipeline. This decision makes the system easier to inspect, easier to run on the course HPC environment, and easier to evaluate with explicit source-level diagnostics.

Methods

Data

The document collection contains seven local HTML files downloaded from PISRS. Table 1 lists the legal sources used in the current experiments. The files are kept in the project downloads directory for the local and HPC runs. Generated chunks, indexes, and evaluation files are written separately to `data/` and `logs/`.

The ingestion module supports HTML, PDF, Markdown, and plain text. For the evaluated run, all source files were HTML. The pipeline removes scripts, styles, and non-visible HTML content with BeautifulSoup, decodes entities, normalizes whitespace, and stores one document record per source file.

Chunking

We implemented and evaluated two chunking strategies.

Table 1. Legal documents used in the current RAG evaluation.

Identifier	Document
ZDDV-1	Value Added Tax Act and implementing rulebook
ZDoh-2	Personal Income Tax Act
ZDDPO-2	Corporate Income Tax Act and implementing rulebook
ZDavP-2	Tax Procedure Act and implementing rulebook

Fixed chunking The baseline splits each document into overlapping character windows of 1,200 characters with 200 characters of overlap. This is simple and robust, but it ignores the article structure of legislation.

Legal chunking Version 2 detects article headings such as `3. člen` and `113.a člen`. The chunker splits text into legal sections and adds metadata for the source file, law identifier, document role, article number, article title, and section position. Long articles are still split into subchunks, but continuation chunks receive an explicit law/article prefix.

The legal chunking strategy was added because the first evaluation showed that fixed chunks often retrieved passages from the correct general topic but not the exact legal article. Article metadata also makes evaluation more meaningful: after changing chunking, exact old chunk identifiers are no longer stable, but source and article hits remain interpretable.

Embeddings and Dense Retrieval

All chunks are embedded with a multilingual MiniLM sentence-transformer. The resulting vectors are normalized and stored in a FAISS `IndexFlatIP` index. Since both document and query vectors are normalized, inner product search is equivalent to cosine similarity. For a user question q and chunk c_i, dense retrieval ranks chunks by

$$s_{\text{dense}}(q, c_i) = \frac{f(q) \cdot f(c_i)}{\|f(q)\| \|f(c_i)\|}, \tag{1}$$

where $f(\cdot)$ is the embedding model. The default final context size is `top_k=3`.

Hybrid Retrieval

The Version 2 retriever uses dense retrieval as a candidate generator and then reranks candidates. FAISS first returns a larger set of candidates, `candidate_k=100`. Each candidate is then scored with

$$s(q, c) = s_{\text{dense}}(q, c) + 0.25 \cdot s_{\text{lexical}}(q, c) + s_{\text{law}}(q, c). \tag{2}$$

The lexical component is normalized token overlap between the question and a combined representation of chunk text plus metadata. The law component gives a 0.20 boost when the question mentions or strongly implies a legal act such as ZDDV-1, ZDoh-2, ZDDPO-2, or ZDavP-2 and the chunk metadata matches that law. The purpose is to reduce cases where the dense model retrieves semantically similar but legally wrong material from another act.

Generation

The answer generator loads the local course model from the shared HPC project directory. Retrieved chunks are rendered into a prompt with source filename, chunk ID, retrieval score, law ID, article number, and article title. We evaluated two prompt variants that completed in the available logs:

Baseline prompt Instructs the model to answer only from retrieved context and cite sources.

Strict prompt Adds stronger instructions to prefer the chunk whose legal act and article directly answer the question, ignore merely related chunks, and say that the answer is not found if the exact rule is missing.

The strict prompt was introduced after the first evaluation showed that the generator often produced fluent answers from related but wrong retrieved passages. The prompt comparison is controlled because both prompt variants use the same retrieved chunks for every question in the Version 2 run.

Evaluation

The evaluation set contains 30 Slovenian tax questions. Each case contains a question, expected source files, expected legal locations, expected phrases, previous expected chunk IDs, and a reference answer. The questions cover VAT definitions and obligations, personal income tax, corporate income tax, and tax procedure.

We report automatic retrieval metrics and manual generation scores:

Source hit@3 At least one retrieved chunk comes from an expected source file.

Article hit@3 At least one retrieved chunk matches the expected legal article.

Phrase hit@3 All expected key phrases are present in the retrieved top-3 context.

Context relevance Manual 0–2 score for whether retrieved context contains the exact rule.

Faithfulness Manual 0–2 score for whether the generated answer is supported by retrieved context.

Correctness Manual 0–2 score for whether the generated answer answers the question correctly.

Exact old chunk ID hit is still available in the logs for debugging, but it is not reported as a main Version 2 metric because article-aware chunking changes chunk boundaries and therefore changes chunk IDs.

Results

Retrieval

Table 2 compares the old baseline and the Version 2 retrieval pipeline. The original dense/fixed setup retrieved the correct source for 8 of 20 questions, but never retrieved the exact expected article or all expected phrases. Legal chunking and hybrid reranking substantially improved source-level retrieval and produced exact phrase coverage for 7 of 20 questions.

The strongest improvement is source hit@3, which rises from 40.0% to 95.0%. This shows that the law-source boost and metadata-aware reranking usually select the correct act. The weaker result is article hit@3, which reaches only 30.0%. This means that the system often finds the correct law but still retrieves a related article instead of the exact definitional article. For example, several ZDDV-1 definition questions retrieved rulebook articles or later VAT procedure articles rather than the expected core definition articles.

Prompt Comparison

The baseline and strict prompts used identical retrieved top-3 chunks in all 20 Version 2 prompt cases. Therefore differences in generated answers are caused by the prompt, not by retrieval variation. Manual scoring is shown in Table 3.

The strict prompt gives a small improvement in correctness, but the change is not large. The main reason is that generation quality is bounded by retrieval quality. When the retrieved context contains the exact rule, both prompts can produce a good answer. When retrieval contains only related material, both prompts either answer the wrong legal question or correctly say that the exact answer was not found. The strict prompt is more cautious, but it cannot recover missing articles.

Qualitative Error Analysis

The evaluation logs show four common error patterns.

1. **Correct law, wrong article.** The retriever often identifies the correct legal act but chooses a related article. This occurs in questions about the object of VAT, taxable persons, tax base, and tax rates.
2. **Rulebook instead of statute.** Several VAT questions retrieve implementing rulebook provisions that contain the same terms but not the primary statutory definition.
3. **Cross-law semantic confusion.** Questions about resident and non-resident obligations can retrieve similar wording from ZDDPO-2 when the question asks about ZDoh-2.
4. **Faithful wrong answers.** The language model usually follows the retrieved context. This is positive for faithfulness, but it means wrong retrieval directly produces wrong answers.

Successful cases include ZDoh-2 source of income, ZDDPO-2 transfer pricing, ZDavP-2 taxpayers, and the right to information. In these cases, the correct article is retrieved and the generated answers are mostly complete. Failed cases include several definition questions where the expected answer is in an early article, but retrieval selects later articles containing overlapping tax vocabulary.

Discussion

The Version 2 pipeline is a clear improvement over the initial baseline. Article-aware chunking gives the index more legally meaningful units, and hybrid reranking significantly improves source-level retrieval. The system is now able to demonstrate the full RAG loop on the HPC: ingest documents, build an index, retrieve legal context, generate answers with a local model, and evaluate the results with structured logs.

However, the current system is not yet reliable enough for production legal question answering. Article hit@3 is only 30.0%, and manual correctness remains below 1.0/2. The main bottleneck is retrieval precision, not model fluency. The generator frequently produces answers that are faithful to retrieved context, but if the context is not the exact article, the answer is legally wrong or incomplete.

The most important next improvement is to add a stronger sparse retrieval component. The current hybrid method reranks dense candidates, so it cannot recover an exact article if that article is not included among dense candidates. A BM25-style retriever over all chunks, combined with dense retrieval, would likely improve definition questions where exact legal wording matters. A second improvement is to index article titles separately and boost matches between question terms and article titles. Finally, the evaluation set should be updated so that expected chunks are defined for the legal chunking scheme, or replaced entirely by source+article+phrase expectations.

Conclusion

We built a reproducible RAG prototype for Slovenian tax legislation and evaluated it on 20 manually designed questions. The baseline dense retriever was too weak for legal question answering, but Version 2 article-aware chunking and hybrid reranking improved source hit@3 to 95.0%. The remaining failures show that legal RAG requires exact article retrieval, not only topical similarity. Future work should focus on sparse retrieval, stronger article matching, and stricter answer abstention when the exact legal rule is not present.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *CoRR*, abs/2005.11401, 2020.

[2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *CoRR*, abs/2106.09685, 2021.

Table 2. Automatic evaluation metrics. Generation scores are marked n/a for retrieval-only runs because no answer was generated.

Run	n	Source@3	Article@3	Phrase@3	Context/2	Faith./2	Corr./2
V1 old dense run	30	0.4	0.1	0.15	0.10	1.0	0.25
V2 fixed dense	30	0.45	0.15	0.20	0.40	1.05	0.30
V2 legal hybrid retrieval	30	0.90	0.30	0.35	1.30	1.15	0.40
V2 legal hybrid + baseline prompt	30	0.90	0.35	0.35	1.30	1.45	0.65
V2 legal hybrid + strict prompt	20	0.95	0.40	0.35	1.30	1.05	0.80

Table 3. Manual generation assessment over 20 questions. Scores use a 0–2 scale.

Metric	Baseline prompt	Strict prompt
Context relevance	1.20	1.25
Faithfulness	1.45	1.55
Correctness	0.60	0.65

[3] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.